



Caruca v2

LLM-based specification mining for opaque shell commands

Scientific vs. Engineering Contributions

Version 1.1 — corrected 22 September 2026

SOURCE `ai_docs/analysis/scientific_vs_engineering_contributions.md`

GENERATED September 22, 2026 at 4:34 PM

PAGES 17

CARUCA V2 — RESEARCH PROJECT

Table of contents

Scientific vs. Engineering Contributions — Framing v2 for Publication

Part 1 — What “not scientifically interesting” means

The three questions every reviewer asks

The operational difference

What the data says reviewers reward

The rubric reviewers will actually use

Part 2 — A result v1 discovered and did not claim

Part 3 — Candidate scientific contributions

S1. How far do hand-written specifications hold, and what happens where they don't?

S2. Does the LLM understand the documentation, or remember it?

S3. Give a mined specification a semantics — what does it actually guarantee?

S4. Falsification-driven mining, and whether it converges

S5. Treat LLM disagreement as signal, not noise

S6. Where does documentation diverge from behavior?

S7. Measure downstream utility, not specification fidelity

S8. Is the method about shells, or about opaque components?

S9. A ground-truth generator, not a ground-truth set

Part 4 — Necessary work that will not count as a contribution

Part 5 — Recommendation

Paper A — the resubmission spine: What does a mined specification guarantee?

Paper B — the fast, standalone one: Does the model read the docs, or remember them?

The conversation to have with Greenberg

Sources

Scientific vs. Engineering Contributions — Framing v2 for Publication

Version 1.1 — note added 22 September 2026. No claim in this document was found to be wrong; its account of the paper’s PaSh comparison (§“the right call for the question the paper is asking”) is the one the rest of the repository has now been corrected to match. Two updates: the three “cautions before building on it” were since **resolved** by `e0_artifact_pinning.md` — the branch worry is closed, and there is no hand-written `cp` annotation for the generated one to agree with — and where the paper’s 116/120 is cited, note that the shipped artifacts score 78/116 by Caruca’s own instrument (`white_paper_addendum.md` §3).

A working answer to a question Prof. Greenberg has raised repeatedly: which v2 directions constitute a *scientific* contribution, and which are product/engineering improvements that will not survive review at a top-tier venue no matter how well executed. Written for the resubmission objective — the Caruca paper was rejected from its original venue, and the plan is to revise and resubmit incorporating v2’s findings.

Two anchors for everything below:

- The paper, cited throughout as **Lamprou et al., “Caruca: Effective and Efficient Specification Mining for Opaque Software Components,” arXiv:2510.14279** (Oct 2025) — full text at `ai_docs/refs/caruca_white_paper.md`.
- v1’s code, at `~/stevens/caruca/` (WSL) or `~/dev/stevens/caruca/` (Mac); remote `binpash/caruca`, commit `d80324073`.

Companion documents: `evaluation_gaps.md` catalogues 14 coverage gaps — Part 4 below explains which of them can be converted into results and which cannot.

`v2_fidelity_to_v1.md` covers mechanism-level correspondence.

Part 1 — What “not scientifically interesting” means

The three questions every reviewer asks

Greenberg has written about this on his own blog, quoting Laurie Tratt: a proposal must answer

1. **What is the problem being tackled?**
2. **Why is the problem worth being tackled?**
3. **What is the insight that makes tackling the problem plausible?**

He adds that questions 2 and 3 carry equal weight with question 1, and that in programming languages students routinely mistake *technical difficulty* for *intellectual contribution* — “proofs aren’t worth the wildly overengineered LaTeX macros they’re written in if the properties they ensure don’t matter to anyone.”

Read the feedback through question 3. “V2 uses 3,000 fewer lines” and “v2 runs faster” are answers to a question nobody asked. They are properties of the artifact. An artifact is not a finding.

The operational difference

	ENGINEERING CLAIM	SCIENTIFIC CLAIM
Shape	“Our system does X well”	“X is true of the world, and we didn’t know that”
Test	Does it work?	Could it have come out the other way?
If it fails	Fix it and try again	Publish the negative result
Transfers to	Users of the tool	Anyone working on this class of problem

The middle row is the sharpest filter. Before proposing anything for v2, ask: *what result would have falsified this, and would that result have been interesting too?* “We reduced lines of code by 47%” fails — nobody would have published “we reduced it by 3%.” “The LLM’s accuracy drops from 97% to 40% when we rename the commands” passes, because “it doesn’t drop” would have been equally worth knowing.

What the data says reviewers reward

Mary Shaw classified every ICSE 2002 submission by question type, result type, and validation type, then cross-tabulated against acceptance. The pattern has been stable for two decades.

By result type (accepted ÷ submitted):

RESULT TYPE	RATIO
Empirical model — a predictive model built from real data	25%
Tool or notation	20%
Procedure or technique	18%
Analytic model — a structural model permitting formal analysis	15%
Specific solution / prototype / judgment	15%
Qualitative or descriptive model	8%
Report — “interesting observations, rules of thumb”	0%

By validation type:

VALIDATION	RATIO
Experience — the result was actually used, with data	24%
Analysis — formal derivation, or a designed statistical experiment	23%
Example — worked demonstration	20%
Evaluation against stated criteria	5%
Persuasion — “I thought hard about this and I believe...”	0%

Two zeros: observations without a model, and arguments without evidence. Note that *Evaluation* scores 5% — measuring your own system against criteria you chose is nearly as weak as not validating at all. That is exactly the category “v2 is faster and smaller than v1” falls into.

Analytic model and *Empirical model* are the two result types where the insight lives in the idea rather than the code. Those are what the feedback is asking for.

The rubric reviewers will actually use

The ACM SIGPLAN Empirical Evaluation Checklist (Berger, Blackburn, Hauswirth, Hicks, 2018) is cited in PLDI's call for papers and is the closest thing to an explicit rejection rubric. Four of its seven categories bear directly on Caruca:

- **Claims not appropriately scoped** — the checklist's own example is “works for all Java when evaluated on a subset.” Caruca's abstract claims “opaque software components” and general applicability; the evaluation is GNU coreutils on Linux. This is a live objection.
- **Indirect or inappropriate proxy metric** — the 97.78% invocation-coverage figure is a proxy for “the specification is good enough to use.” Coverage is not utility.
- **Tested on training data** — “when a system aims to be general but was developed with close consideration of specific examples.” Coreutils man pages are in GPT-4o's training set. A reviewer will think of this immediately, and the paper does not address it.
- **Fails to acknowledge limitations** — the checklist's notes are unusually blunt: “For science to progress, we need to be honest about what we have achieved.”

Part 2 — A result v1 discovered and did not claim

Hand-written command specifications are approximations, and the people who write them know it. The PaSh annotation for `grep` at `benchmarks/annotations/pash/grep.json` carries its authors' own caveats inline, in comment fields next to the cases they qualify:

“This doesn't work if the pattern is given with the `-e`, `-f` flags”

“In this case this reads directories and that is why we conservatively assume it is side-effectful. It is possible that this could be made more precise to be stateless.”

“This isn't *quite* right—we need to identify a pattern somewhere and exclude it from `input`.”

That is careful, honest annotation practice — the authors flagging the exact places their model is coarser than the command. It is also an open question they had no way to close: until Caruca, there was no mechanism for checking an annotation against a command's

actual behavior across the whole invocation space, so the cost of those approximations could be acknowledged but not measured.

Caruca can measure it. The paper reports an instance in passing, in the section evaluating specification correctness (section 7.1), while explaining its methodology:

PaSh’s specifications “do not always generalize outside of their evaluation benchmark suite; for example, `grep` is marked as always stateless in the original specifications, but in reality it is not if invoked with the `-c` flag... Similar issues affect the ground-truth specifications provided alongside PaSh for the commands `ps` (marked as stateless, but in reality being side-effectful) and `cp` (marked as side-effectful, but in reality being pure).”

The paper then compares only against the invocations inside PaSh’s own benchmark suite, and reports 52 of 52. **That is the right call for the question the paper is asking.** If the claim is “Caruca reproduces the hand-written ground truth,” the ground truth has to be held fixed, and restricting to the invocations where it is known to hold is the conservative and defensible choice.

The observation for the resubmission is that a *second* question was sitting in the same data, and it is the more interesting one. Not “does Caruca agree with the annotations?” but **“how far do the annotations hold, and what happens downstream where they don’t?”** Answering it needs exactly the machinery v1 already built, and it changes the role of the hand-written specifications from the yardstick to the object of study.

This is a sense in which v1 under-claimed rather than over-claimed. The automation is what made the observation possible at all; it entered the paper as a methodological aside because the paper was answering the first question. The resubmission can answer the second, and the annotation comments above are the argument that the field would welcome it — the annotators asked this question of themselves first.

One caution before building on it. Checking against the repository’s `main` branch raised three things that need resolving. The sample annotation in `benchmarks/annotations/pash/grep.json` does handle `-c`, as its first case. The generated `cp` annotation on `main` agrees with the hand-written one rather than contradicting it. And the paper’s runs came from the `caruca-experiments` branch, not `main`. The likeliest explanations are benign, and one is actively good news: the annotations may have been corrected upstream after Caruca surfaced the divergences, which would mean the findings were already adopted. Either way it needs to be pinned and documented rather than assumed. That is experiment E0 in [experiment_designs.md](#), and it blocks the rest.

Part 3 — Candidate scientific contributions

Ranked by (new knowledge produced) $\times$ (feasibility with infrastructure that already exists). Each gives the claim, why it is science rather than engineering, and the experiment.

S1. How far do hand-written specifications hold, and what happens where they don't?

Claim: across PaSh, POSH, Shellcheck, and Shseer, hand-written command specifications diverge from observed command behavior at rate R , falling into taxonomy T , of which K are soundness-critical — they can make the consuming system produce a wrong answer, not merely miss an optimization.

Why it's science: a falsifiable claim about artifacts the field depends on. If R turns out to be near zero, that is *also* a result — it validates the crowd-sourcing premise those papers propose. Result type: empirical model plus findings. Validation: analysis and experience.

Experiment: run Caruca as a *differential oracle* against every hand-written annotation in all four systems, not just the ones inside each system's own benchmark suite. For each disagreement, construct a **witness** — a real shell script where PaSh with the hand-written specification and PaSh with the observed behavior produce different output. Classify each divergence: over-general flag handling, missing flag-conditional behavior, imprecise purity class, stale across command versions. Report the soundness-critical subset separately.

What it changes: Caruca becomes the *instrument* rather than the *claim*. The telescope is not the paper; what you see through it is.

Risk to manage: those systems' authors are prospective co-authors on the resubmission. The honest framing is also the diplomatic one — manual annotation cannot avoid these errors at scale, which is the paper's own motivating argument, now supported by evidence rather than assertion.

S2. Does the LLM understand the documentation, or remember it?

Claim: Caruca's syntax-inference accuracy on GNU coreutils (116 of 120 exact command-level matches; 99.7% at the argument level) is partly recall of memorized documentation rather than comprehension. Measure the gap.

Why it's science: the answer is genuinely unknown, it could go either way, and it determines *when the method generalizes* — the question the paper leaves open and reviewers will press on. It also converts the most likely rejection reason into a contribution.

Experiment — three arms of increasing severity:

1. **Held-out:** commands released after the model's training cutoff, plus obscure third-party tools with low web presence.
2. **Mutated documentation (the strong test):** take a man page and consistently rename the command and its flags — `cat` → `zorp`, `-n` → `--enumerate` — while preserving the described semantics. Recall now *actively hurts*; only reading helps. Then a harder variant: keep the names but permute the semantics, so `-n` does what `-s` used to. If accuracy holds, the model is reading. If it snaps back to the real `cat` specification, it is remembering.
3. **Synthetic tools:** mechanically generate CLI programs with randomized flag sets, then generate their man pages from the implementation. Ground truth is correct *by construction*, uncontaminated, and unlimited — which also relieves the annotation bottleneck (see S9).

Deliverable: an accuracy-versus-contamination curve. Publishable on its own at ISSTA, FSE, or ICSE even if nothing else in v2 lands.

S3. Give a mined specification a semantics — what does it actually guarantee?

The most likely to land with Greenberg, and the code for it already exists.

The observation: v1 already records *why* each match was accepted.

`caruca/src/caruca/ir/syntax.py` defines a hierarchy — `Exact` versus `Extrapolated`, with `AliasStrategy`, `ShortFlagSplat`, `DefaultArg`, `ArityStrategy`, and `Agnosticism` splitting into `PathAgnosticism`, `IntAgnosticism`, and `StringAgnosticism`.

`caruca/src/caruca/oracle.py` collects them per invocation and exposes an `--exclude` filter over them. Then `eval/user-scripts/completeness_accumulate.sh:10-14` builds the paper's coverage figure by peeling those filters off one tier at a time: `no_assumption`, then `flag_match`, then `path_match`, then `int_match`, then `string_match`.

That is a lattice of increasingly strong assumptions, and it is implemented with `grep -v`. The paper reports its tiers — 10% exact, 18% with path and integer normalization, 94% with string normalization, 97.78% with arity — but never says what any tier *means*. It is a formal structure with no formal content.

Claim: every mined specification carries a *validity domain* — the set of executions for which it is justified — derived compositionally from the extrapolation steps used to build it. Each extrapolation is an explicit semantic assumption: `StringAgnosticism` assumes argument

content does not affect the property being inferred; `ArityStrategy` assumes behavior is uniform in the number of repeated arguments. Downstream soundness then becomes a theorem conditional on those assumptions rather than a hope.

Why it's science: this is an analytic model — the highest-status result type at a PL venue, and the thing the paper conspicuously lacks. It answers the question a PLDI or OOPSLA reviewer asks first: *what does it mean for your specification to be correct, given that you showed the hand-written ground truth is itself approximate?* At present there is no answer.

What it buys, concretely:

- The 97.78% figure becomes *typed* rather than *blended* — the presentational fix already flagged as gap 13 in `evaluation_gaps.md`, now with a reason behind it instead of a disclaimer.
- PaSh can be handed a policy: *only use specifications whose validity domain is at least this strong* — and you can measure what that policy costs in missed optimizations. That is a real soundness-versus-coverage tradeoff curve.
- The assumptions become **testable**. `StringAgnosticism` is a claim about commands. For what fraction of coreutils does argument content actually change the inferred property? That single experiment converts the weakest step in the coverage argument into a measured quantity.

S4. Falsification-driven mining, and whether it converges

The reframe: v1's retry loop fires on *schema validation* failures — the LLM produced malformed output, so the previous output and the error are fed back, up to three attempts. That is error handling. Replace it with a loop on *behavioral counterexamples*: the LLM proposes a hypothesis specification, the executor searches for an execution that contradicts it, the counterexample repairs the hypothesis, repeat.

Why it's science: it turns questions about speed into questions about *query complexity*, which is a recognized theoretical frame — Angluin's L^* , counterexample-guided inductive synthesis, active automata learning. The real questions become: does it converge? in how many rounds? does counterexample *selection* matter — random versus maximally-distinguishing? how many executions does adaptive querying need against exhaustive enumeration over the flag space?

This is how “v2 is faster” becomes publishable. Not “3× speedup” but: exhaustive enumeration is exponential in flag count, which is why v1 must cap exploration at four flags (the pruning described in the paper's computational-cost section, 7.4); adaptive querying

needs $f(n)$ executions empirically; here is the curve, and here are the commands that break it. Same work, different claim. The four-flag cap stops being a limitation and becomes a baseline you beat for an articulable reason.

S5. Treat LLM disagreement as signal, not noise

The weak version, to avoid: “we sampled five times and report the variance.” That is quality assurance. It answers “is our tool reliable,” which is engineering.

The strong version: sample the specification k times. The **intersection** across samples is a candidate under-approximation; the **union** is an over-approximation; the **disagreement set** is exactly where the model is uncertain. Claim: disagreement localizes error better than the model’s self-reported confidence, and spending execution budget on the disagreement set beats spending it uniformly at equal total cost.

Why it’s science: a mechanism with an ablation, falsifiable, and general beyond shell to any “LLM proposes, oracle disposes” pipeline. It also converts nondeterminism from a caveat into the engine. This is the scientifically load-bearing version of gap 9 in [evaluation_gaps.md](#).

S6. Where does documentation diverge from behavior?

Claim: an empirical study, at population scale, of how often and in what ways a command’s documentation misdescribes its behavior. Categories: undocumented flags, wrong arity, semantics that silently changed across versions, and behavior that differs between GNU, BSD, and busybox builds of the same nominal command.

Why it’s science: a finding about the world, judged on truthfulness and interestingness. Caruca is the only machine that can do it at scale — documentation goes in one side, observed behavior comes out the other, and the diff is the result. Filed bug reports are strong evidence of impact.

Bonus: this redeems the paper’s cross-platform motivation. The introduction motivates the whole problem with “differences across command versions on Linux, macOS, BSD”; the evaluation never tests it (gap 3 in [evaluation_gaps.md](#)). Running the *same generated specification* against BSD and busybox builds turns a motivational gesture into a measured divergence taxonomy — and makes the Linux-only tracing stack a scoping decision you report rather than a hole a reviewer finds.

S7. Measure downstream utility, not specification fidelity

The problem with the current metric: Caruca is scored against hand-written specifications whose limits S1 measures. Perfect agreement with an approximate oracle is not, by itself, a good outcome.

Claim: with mined specifications, PaSh and Shseer can analyze N real-world scripts they previously could not, yielding measured speedups and B newly-found bugs — and, critically, mined specifications cause a divergent result in E cases. E is the number that decides whether any of this is deployable.

Why it's science: “does automating X actually unlock Y?” is the assumption the entire research program rests on, and it has never been tested. It also replaces a proxy metric (coverage percentage) with a direct one, which is explicitly what the SIGPLAN checklist asks for.

Experiment: the corpus infrastructure already exists — `eval/command-invocations.txt` and `eval/pash-scripts/` in v1. Extend the corpus beyond standalone shell scripts to GitHub Actions `run:` blocks and Dockerfile `RUN` lines (gap 4 in `evaluation_gaps.md`) — different populations with different flag distributions — and report per-population results. Report E honestly and prominently; a small nonzero E with a diagnosis is a much stronger paper than a suspicious zero.

S8. Is the method about shells, or about opaque components?

The title says “opaque software components.” The evaluation says GNU coreutils on Linux. That gap is a scoping violation a reviewer will name.

Claim: the pipeline — documentation to structured interface, generate configurations, execute under interposition, derive properties — transfers to a component class that is not a Unix command. Candidates in ascending cost: BSD and busybox variants (cheapest, and it doubles as S6), Windows PowerShell cmdlets, Python library functions behind a generated CLI shim, REST endpoints lacking an OpenAPI description.

Why it's science: one genuinely different domain converts a *specific solution* into a *technique* — the modal accepted result type in Shaw's data — and retires the over-claim in the abstract by making it true.

S9. A ground-truth generator, not a ground-truth set

Two graduate students spent 80 person-hours annotating 120 commands, and reconciled 16 discrepancies between themselves. That is the bottleneck on every future evaluation, and the ground truth is itself uncertain.

Claim: synthetic CLI tools with mechanically generated documentation provide ground truth that is correct by construction, unlimited in quantity, and provably uncontaminated.

Why it's science: infrastructure becomes a contribution when it enables claims nobody else can make. It is the enabling condition for S2, and it gives the community a reusable benchmark — increasingly valued at artifact-evaluated venues.

Part 4 — Necessary work that will not count as a contribution

Do these; just don't claim them.

V2 WORK ITEM	WHY IT ISN'T A CONTRIBUTION	SALVAGE
Lines-of-code reduction	A property of the codebase. Nobody would publish “we reduced it by 3%.” Also confounded — the paper’s 6,520 LOC is roughly half generated specification <i>data</i> , leaving ~3,456 lines of hand-written logic as the real baseline (see caruca_v1_loc_baseline).	Drop as a headline; one sentence in the artifact description. The interesting version is S3: hand-written extrapolation rules encode assumptions, and the question is <i>which</i> assumptions, not how many lines.
Wall-clock speed	Machine-dependent, and “faster” is expected of a rewrite.	Reframe as query complexity (S4), or as an accuracy-versus-budget Pareto curve.
Token and dollar cost telemetry	Measuring your own bill.	Interesting in exactly one framing: the paper claims it “eliminates manual effort” but never prices its own automation against the 80 person-hours it displaces. Make that comparison once, with the cost curve.
Adding fixture axes — symlinks, permissions, empty content, wider integers, environment variables, nested directories, correlated multi-file inputs, richer content types (gaps 1, 5, 6, 7, 8, 10, 12, 14)	Individually, each is “we tested more things.” Reviewers read a list of eight as a to-do list, not a result.	One reframe rescues all eight at once: fixture-axis sensitivity. Ablate each axis and measure <i>which axes change the inferred property, and for what fraction of commands</i> . Adding permissions then produces a finding — “permission state changes observed behavior for X% of coreutils; content type for only Y%” — and tells the field where to spend fixture budget. That is an empirical model.
Running split invocations concurrently to validate	On its own, one more test mode.	Becomes S1’s witness generator — it is how you <i>demonstrate</i> that an imprecise parallelizability

V2 WORK ITEM	WHY IT ISN'T A CONTRIBUTION	SALVAGE
parallelizability (gap 2)		annotation causes real divergence.
Web GUI, PTY terminal, configuration surface, model and decoding-parameter selection	Artifact quality.	Belongs in artifact evaluation, where it genuinely helps. Not in the claims.
Reproducibility check across repeated runs (gap 9)	As stated, quality assurance.	Becomes S5 when disagreement is used as a <i>mechanism</i> rather than reported as a <i>caveat</i> .

Part 5 — Recommendation

Two coherent papers are available. They are not in conflict, and the second is faster.

Paper A — the resubmission spine: *What does a mined specification guarantee?*

- **Motivating finding:** S1 — how far hand-written specifications hold, measured, with witnesses for the cases where the divergence is soundness-critical.
- **Technical core:** S3 — validity domains derived from the extrapolation lattice already implemented in `ir/syntax.py`, giving conditional soundness statements and a typed coverage number.
- **Mechanism:** S4 and S5 — counterexample-driven refinement, with disagreement directing execution budget.
- **Validation:** S7 — downstream utility, and honestly reported downstream divergence.

This keeps everything v1 contributed and adds the layer it was missing: a theory of what the output means. It answers Tratt's third question with an insight — *extrapolation steps are assumptions, assumptions have semantics, and a specification is only as sound as the assumptions used to build it* — rather than with an artifact.

Paper B — the fast, standalone one: *Does the model read the docs, or remember them?*

S2 plus S9, optionally with S6. Smaller, self-contained, currently topical, and publishable independent of whether Paper A lands. A reasonable candidate for a first-author publication early in the PhD.

The conversation to have with Greenberg

Bring him the `grep` / `ps` / `cp` paragraph from the paper's specification-correctness section and one question: *"The annotations the field depends on are approximations, and their authors say so. The paper measured how well we match them. Should the resubmission measure how far they hold instead?"* That is a research question rather than a product question, and it reframes v1's own observation as the starting point rather than as a critique.

Sources

- Mary Shaw, [Writing Good Software Engineering Research Papers](#), ICSE 2003 — question, result, and validation taxonomies with ICSE 2002–2003 acceptance ratios.
- ACM SIGPLAN, [Empirical Evaluation Checklist](#) (Berger, Blackburn, Hauswirth, Hicks, 2018) — cited in the PLDI call for papers.
- Michael Greenberg, [What's hard about grad school?](#) — the Tratt three-question framing, in his own words.
- Lamprou et al., [Caruca: Effective and Efficient Specification Mining for Opaque Software Components](#), arXiv:2510.14279.